

1. ?????????????????? 业务模型

弱网化/社区化是斯凯单机游戏平台的发展方向。斯凯平台会提供一些具有代表性的弱网化模板来快速的改造游戏。这包括已经在广泛推广的积分上传系统。?联网对战系统是第 2 种弱网化方式。

?

联网对战的业务模型非常简单，大致可以分为以下几个业务交互：

?

?

1????????? 上传玩家数据到服务器

1????????? 获取其他玩家的数据

1????????? 模拟对战

1????????? 上传对战结果

1????????? WAP 前端

?

1. 1????????????????? 上传玩家属性到服务器

?

玩家属性是指一个玩家在某个游戏中的数据。例如游戏中的“体力，武力，等级，智力”等属性。

?

在游戏进程加入一些点鼓励玩家将数据上传到服务器。例如在游戏中升级时，可以提示玩家上传属性。?为了鼓励玩家上传，可以给予一定的奖励措施，服务器也可能会给一些奖励，例如奖励服务器端的虚拟货币或者在线道具等（开发中）。

?

第一步 提示玩家

?

当玩家点击“确定”按钮后，游戏将玩家的属性按照格式填好，?调用 SDK 里提供的 API，

?

在 SDK 里，首先会判断玩家是否已经登录过，如果没有登录过，会启动 PASSPORT 插件来提示玩家登录。如果已经登录过，会直接跳过这一步。

?

第二步 登录

?

接下来就是联网/上传数据的部分。?SDK 中提供的 NETPK 在后台工作，游戏需要做一个“上传中。。。 ”的 UI。

?

第三步： 和服务器交互

?

NETPK 插件完成和服务端的交互后，会用 callback 来通知游戏上传的结果。?游戏需要做一个 UI 来显示结果给玩家看。

?

第四步： 上传结果

?

[获得 3K 款]： 服务端有可能会给玩家一些奖励， 终端只需要显示服务端下发的字符串信

息即可， 不需要关心具体的含义。

?

1. 1. 1?????????? 属性数据的设计

由于 NETPK 功能并不知道具体的游戏逻辑，所以也无法知道玩家的属性的具体含义。服务端只是提示一个通用的存储。 目前的设计如下：

字段说明是否可选 title 头衔, ?主要用来在 WAP 平台上显示。可为 NULL20 个 int32 字段保存 Int 类型的数据根据需要进行选择所需的字段 4 个 unicode 字符串?? 【开发中】?根据需要进行选择所需的字段 1 个 byte[] 字段不能使用 int 或者 String 表示的数据，可以用二进制数组。游戏需要自己来编解码。根据需要进行选择所需的字段?

游戏可以根据自身的需求来选择所使用的字段。

上传的属性可以在 WAP 页面中进行展示， 由于服务器无法理解每一个字段的具体含义， 所以需要在服务器端部署每一个字段的具体名称。

例如

字段 WAP 中显示的名称 int32?? 1 等级 int32?? 2 体力 int32?? 3 武力 String?1 官衔?

没有部署名称的字段将不在 WAP 页面上展示， 包括 byte[] 字段也无法显示。

1. 1. 2?????????? 上传 Feed

?游戏中发生的一些重要事件可以上传到服务端的 Feed 系统， 从而在冒泡平台上进行传播，可以极大的提升游戏的曝光度：

例如： 在斯凯的 WAP 页面传播：

?????? 玩家专门宰杀菜鸟在游戏三国志的虎牢关战役中生擒吕布，获得方天画戟！

Feed 的内容可以由开发者自己来指定。 请参考 API 说明。

??????

?

?

1. 2???????????????? 获取其他的玩家的属性

游戏中鼓动玩家发起和其他的玩家的 PK。

?

第一步： 游戏引导玩家发起挑战

?

游戏可以设定一定的选择标准来让玩家找到想要挑战的对手。

?

?

选择好了筛选标准后， 开始调用 SDK 中的 API。?? SDK 封装了如下的逻辑：?加载判断是否已经登录过(同上传时)， 加载 NETPK 功能插件， 并后台获取满足条件的玩家列表和对应的属性数据。 这个过程游戏需要提供 UI 让玩家等待。

?

第二步 后台获取数据

?

玩家列表获取成功后， 通过回调函数来通知游戏进行展示， 游戏需要做一个 UI。实现选择， 上下翻页的功能。

?

第三步选择玩家

1. 3???????????????? 模拟战斗

玩家选择了对手后， 进行到模拟 PK 阶段。 这个阶段完全由游戏自己控制：

自己的昵称可以通过 API 获得。

?

1. 4???????????????? 上传对战结果

如果玩家挑战失败，?那么不需要向服务器报告结果。

?

挑战成功后， 让玩家上传战绩。

?

玩家确定， 再次调用 SDK 中的接口进行上传成绩。 ?游戏可以选择性的为这次战斗添加一条 Feed

????? “玩家 1” 在游戏 “游戏名称” 中挑战 “玩家 2” 胜利， 获得奖励 XXX。

?

?

由于也是异步过程， 游戏需要提供一个 UI 提示玩家等待。

?

?

SDK 会通过 callback 来通知游戏， 上传是否成功。

?

给玩家的奖励， ?有如下两种

本地游戏自身确定，， 例如可以从对手那里获得一个装备。

获得奖励应该考虑到自己和对手的等级差距， 也就是难度越大应该获得奖励越多。 和打 BOSS 是同样的道理。服务器控制完全由服务器来决定。例如赠送虚拟货币， 在线道具等。?

?

在服务端，?会进行一些计算，包括胜者的胜场数量+1， 败者的败场数量+1， 同时会根据规则计算出实力指数。

1. 5???????????????? WAP 前端

在 SDK 中提供 API 来进入到 WAP 前端。

?

?

?

2. ????????????????? 技术框架

?

NETPK 插件： 处理真正的上传属性/Feed/PK 结果， 获取对手列表的消息交互。

PASSPORT 插件： 处理登录/注册流程。

?

为了尽量的封装掉内部的细节， SDK 中提供 lib 来供开发者调用。

?

3. ????????????????? API 说明

3. 1???????????????? 上传属性数据/Feed

函数原型：

?

????? #include “mrc_netpk.h”

extern int32 mrc_netpk_upload_appdata(

MR_Character* owner,

const uint16* feed_desc,

```
MR_Netpk_upload_appdata_CB cb);
```

函数说明:

该函数用于向联网 PK 服务端发送角色信息 AND/OR Feed。在上传前会进行是否登录的逻辑检查。如果已登录则直接上传, 如果未登录, 则调用 passport 插件来进行登录/注册操作。返回的数据由游戏负责释放。

可以同时上传数据数据和 Feed, 也可上传其中的一个。

?

输入参数:

参数类型说明 OwnerMR_Character*指向本地角色信息的指针,

可以为 NULLfeed_descuint16*(utf16-be 编码)预上传的 feed 描述字符串(utf16-be 编码),

也可以为 NULLCbMR_Netpk_upload_appdata_CB 上传更新角色信息后的回调。?

Feed 是由 utf16be 编码的字符串, 可以包含的服务器替换内容为:

NOTE: 这些特殊格式的本身也必须为 utf16be 的编码, 而不是 ascii。

特殊格式说明\${SKYID:<skyid>}服务端会替换成玩家<skyid>的昵称\${CURAPPID}服务端会替换成当前应用的名称\${APPID:<appid>}服务端会替换成游戏<appid>的名称\${CURSKYID}服务端会替换成当前登录玩家的昵称?

?

?

返回说明:

返回值说明 MR_FAILED 失败, 参数传递错误。不再调用 CbMR_SUCCESS 参数传递正确, 正在上传角色信息。将通过 CB 来返回结果。回调说明:

回调函数类型: MR_Netpk_upload_appdata_CB

回调函数参数类型说明 errCodeMR_PKErrorCode 如果为 MR_PKErrorCode_SUCCESS, 则 respData 不为 NULL, len 也不等于 0; 否则 respData 为 NULL, len 为 0respDatachar*从服务端获取到的回复信息 (uint16* (utf16-be 编码)。该数据由游戏释放。

游戏需要显示信息给玩家, 但不需要关心具体的内容。Lenint32 从服务端获取到的数据的长度。?

3. 2???????????????? 获取 PK 对手列表

函数原型:

?

```
?????? #include "mrc_netpk.h"
```

```
?????? int32 mrc_netpk_getPKList(MR_PKFilter filter,
                                const char* customFilter,
                                int32 nOffset,
                                int32 nCount,
                                MR_GetPKList_CB cb)
```

函数说明:

??? 该函数主要用于根据输入的筛选条件, 以及需获取 PK 对手数量从服务端获取 PK 列表。该函数根据输入的筛选条件是否与账户相关而去本地获取历史账号, 如果无法获取或者用户需要登录, 则会调用 passport 插件进入账号辅助界面进行注册或登录, 此时可能需要与服务端交互。在 passport 插件以外无 UI 界面。获取到的列表数据由游戏负责释放。

输入参数:

参数类型说明 FilterMR_PKFilter 预设过滤条件, 当预设过滤条件不为 PKFilter_CUSTOM 时, customFilter 无效。customFilterchar*自定义过滤条件, 仅当 filter 为 PKFilter_CUSTOM

时有效; customFilter 为简单规则的筛选表达式, 支持下面的规则:

1. C(N) = Value???? 等于
2. C(N) <> Value??? 不等于
3. C(N) > Value???? 大于
4. C(N) < Value???? 小于
5. orderby C(N)?desc(asc)? 排序。desc: 降序; asc:升序。
6. and?并且
7. or?? 或者
8. (...) 逻辑判断优先级

Value 为 int32 类型值, $0 \leq N < \text{MAX_PREFIXDATA_COUNT}$ 。

例:?? C1=1000 and (C2>241 or C5<>0)?orderby C1 desc

含义: 体力为 1000 并且(等级大于 241 或者魔法不等于 0)的记录筛选出来, 最后以体力的降序输出。nOffsetint32 选择记录集的第 nOffset 开始。nCountint32 得到 nOffset 之后的 nCount 条记录。考虑到内存等条件, 建议不要一下子获取太多角色信息。CbMR_GetPKList_CB 得到 PK 列表结果或获取 PK 列表失败的回调。?

返回说明:

返回值说明 MR_FAILED 失败, 参数传递错误。MR_SUCCESS 参数传递正确, 正在获取数据。将通过 CB 来异步通知结果。回调说明:

回调函数类型: MR_GetPKList_CB

回调函数参数类型说明 errCodeMR_PKErrorCode 如果为 MR_PKErrorCode_SUCCESS, 则 lst 为从服务端获取到的 PK 列表; 否则 lst 参数无效, 为 NULL。LstMR_CharacterList*从服务端获取到的 PK 列表。?

3. 3???????????????? 上传 PK 结果

函数原型:

```
#include "mrc_netpk.h"
int32 mrc_netpk_commit_pkresult(
MR_PKResult* result,
int32 count,
MR_Character* lstHead,
const uint16* feed_desc,
MR_Netpk_commit_PKResult_CB cb);
```

函数说明:

该函数用于在 PK 结束后上传 PK 结果。在本函数的调用之前应该保证在这之前调用了 mrc_netpk_getPKList 函数。且当次的 PK 结果只能上传一次。在调用该函数接口前会进行是否登录的检查, 如果未登录, 则直接返回 MR_FAILED; 否则对游戏提交的该次 PK 结果进行上传。

输入参数:

参数类型说明 ResultMR_PKResult*MR_PKResult 结构体指针, 包含胜利者以及失败者的 skyid。CountInt32 游戏希望更新的角色数量。LstHeadMR_Character*角色信息数组头指针 Feed_descUint16*Feed 字符串 (utf16-be 编码)。CbMR_Netpk_commit_PKResult_CB 上传 PK 结果信息后的回调。 ?

返回说明:

返回值说明 MR_FAILED 失败, 参数传递错误。MR_SUCCESS 参数传递正确, 正在上传 PK 结果信息。回调说明:

回调函数类型: MR_Netpk_commit_PKResult_CB

回调函数参数类型说明 errCodeMR_PKErrorCode 如果为 MR_PKErrorCode_SUCCESS, 则 respData 不为 NULL, len 也不等于 0; 否则 respData 为 NULL, len 为 0respDatachar*从服务端获取到的回复信息 (uint16* (utf16-be 编码)。该数据由游戏释放。Lenint32 从服务端获取到的数据的长度。?

3.4???????????????? 取消上传/更新/获取 PK 信息

函数原型:

```
#include "mrc_netpk.h"
int32 mrc_netpk_cancel(void);
```

函数说明:

在联网 PK 的异步操作中, 如调用 mrc_netpk_getPKList/ mrc_netpk_upload_apdata/ mrc_netpk_commit_pkresult 操作回调未返回时, 程序主动停止这个过程, 相应的回调函数将马上被调用, errCode 返回 MR_PKErrorCode_UserCancel。

输入参数:

无

返回说明:

返回值说明 MR_SUCCESS? ?

回调说明:

无

?

?

3.5???????????????? 获取当前登录用户的 skyID

函数原型:

?

```
#include "mrc_passport.h"
const char* mrc_passport_getCurSkyID(void);
```

函数说明:

如果本地用户登录了以后, 则该函数立即返回本地登录用户的 skyid; 否则返回值为 0。

输入参数:

无

返回说明:

返回值说明当前登录用户的 skyID?0 如果本地用户没有登录, 则返回 0 回调说明:

无

?

?

3.6???????????????? 获取当前登录用户的 NickName

函数原型:

```
#include "mrc_passport.h"
const char* mrc_passport_getCurNickName(void);
```

函数说明:

如果本地用户登录了以后, 则该函数立即返回本地登录用户的 nickname; 否则返回值为 NULL。

输入参数:

无

返回说明:

返回值说明当前登录用户的 nickNameUtf16be 编码, \x00\x00 结束 NULL 如果本地用户没有登录, 则返回 NULL 回调说明:

无

?

?

3. ?????????????????? 显示 WAP 页面

?

```
#include "mrc_netpk.h"
```

```
// 结束回调
```

```
typedef void (*MR_Netpk_openWap_CB)(int32 result);
```

```
int32 mrc_netpk_openWap(MR_Netpk_openWap_CB cb);
```

?

4. ?????????????????? 数据结构

?????? 对于在联网 PK 插件中所涉及到的特定数据类型说明如下。

4.1 ?????????????????? MR_Character

原型:

```
typedef struct
```

```
{?????
```

```
??????? uint32 ctrl;????????????????????????????????????????
```

```
??????? int32?win;????????????????????????????????????????
```

```
??????? int32?lost;????????????????????????????????????????
```

```
??????? char?identify[MAX_IDENTIFY_LEN];??????
```

```
??????? int32?StrengthValue;?????? ???????????????????
```

```
??????? int32 sex;????????????????????????????????????????????????????????????????
```

```
??????? int32 bitmap;????????????????????????????????????????????????????????????
```

```
??????? int32 prefixData[MAX_PREFIXDATA_COUNT];
```

```
??????? uint8 *customData;????????????????????
```

```
??????? int32 dataLen;????????????????????
```

```
??????? int32 skyid;??
```

```
??????? uint16 nickName[MAX_NICKNAME_LEN+1];???
```

```
??????? uint16 title[MAX_TITLE_LEN+1];????????
```

```
}MR_Character;
```

说明:

??? 如下是关于 MR_Character 成员的说明。

?

?

由于上行和下行消息公用了同一个结构, 所以某些字段只在下行时有效。

?

名称类型说明上行下行 ctrluint320 表示终端控制 PK 结果奖励?1 表示服务端控制 PK 结果奖励?Ywinint32 历史胜利场数?Ylostint32 历史失败场数?Yidentifychar[]服务器对终端的描述?YStrengthValueint32 综合实力指数, 由服务器算法控制.?Ysexint32 角色性别?Ybitmapint32 用于表示下面的 bitmap 是否使用, 1 表示有, 0 表示没有

YYprefixDataInt32[] 含 20 个元素的数组，各个元素的含义可自己定义。

如定义：prefixData[0] 存放体力

?????? prefixData[1] 存放等级

各个单元的缺省值为 0YYcustomDataInt8*终端自定义数据 YYdataLenInt32 自定义数据长度 YYSkyidInt32 玩家 skyid?YnickNameuInt16[] 昵称，用 utf6-be 编码?YtitleuInt16 头衔，用 utf6-be 编码 YY4.2????????????????? MR_CharacterList

原型：

```
typedef struct tagCharacterList
{
    ??? MR_Character *pCharacter;
    ??? struct tagCharacterList *pNext;
}MR_CharacterList;
```

说明：

名称类型说明 pCharacterMR_Character*存放当前节点的角色信息 pNextMR_CharacterList*指向下一个角色信息 4.3????????????????? MR_PKFilter

原型：

```
typedef enum
{
    ?? MR_PKFilter_STRENGTH = 0, ??? ????
    ?? MR_PKFilter_SIMILAR, ??????????????
    ?? MR_PKFilter_FIGHTWITHME, ??????????
    ?? MR_PKFilter_FEMALE, ?? ??????????????
    ?? MR_PKFilter_DAY_CHAMPION, ??? ???
    ?? MR_PKFilter_WEEK_CHAMPION, ?? ????????
    MR_PKFilter_CUSTOM?? ??????????
}MR_PKFilter;
```

说明：

名称数值说明 MR_PKFilter_STRENGTH0 综合实力最强 MR_PKFilter_SIMILAR1 实力相近 MR_PKFilter_FIGHTWITHME2 最近挑战过自己 MR_PKFilter_FEMALE3 女性玩家 MR_PKFilter_DAY_CHAMPION4 日冠军 MR_PKFilter_WEEK_CHAMPION5 周冠军 MR_PKFilter_CUSTOM6 自定义 4.4????????????????? MR_PKErrorCode

原型：

```
typedef enum
{
    ??? MR_PKErrorCode_SUCCESS = 0, ??????
    ??? MR_PKErrorCode_UserCancel, ??????
    ??? MR_PKErrorCode_DIAL, ??????????????
    ??? MR_PKErrorCode_Network, ??????????
    ??? MR_PKErrorCode_InvalidUser, ??
    ??? MR_PKErrorCode_BlackListUser, ???
    ??? MR_PKErrorCode_FormatError, ???
    ??? MR_PKErrorCode_NotEnoughMem,
    ??? MR_PKErrorCode_OtherError??????
}MR_PKErrorCode;
```

说明:

名称数值说明 MR_PKErrorCode_SUCCESS0 成功 MR_PKErrorCode_UserCancel1 用户取消
MR_PKErrorCode_DIAL2 拨号错误 MR_PKErrorCode_Network3 网络错误
MR_PKErrorCode_InvalidUser4 无效用户 MR_PKErrorCode_BlackListUser5 黑名单用户
MR_PKErrorCode_FormatError6 格式错误 MR_PKErrorCode_NotEnoughMem7 内存不够
MR_PKErrorCode_OtherError8 其他错误 ?

4.5???????????????? MR_PKResult

原型:

```
typedef struct
{
    ????? uint32 winner;???
    ????? uint32 loser;???
}MR_PKResult;
```

说明:

名称类型说明 WinnerUint32 胜者的 skyid, 也就是自己的 skyid。通过 getCurSkyID() 获得
LoserUint32 输者的 skyid4.6???????????????? MR_PassportErrorCode

原型:

```
typedef enum
{
    ????? MR_PassportErrorCode_SUCCESS = 0, ?????????? /* 成功???????????? */
    ????? MR_PassportErrorCode_UserCancel, ?????????? /* 用户取消???????? */
    ????? MR_PassportErrorCode_DIAL, ?????????????? /* 拨号错误???????? */
    ????? MR_PassportErrorCode_Network, ?????????? /* 网络错误???????? */
    ????? MR_PassportErrorCode_InvalidUser, ?? /* 无效用户???????? */
    ????? MR_PassportErrorCode_BlackListUser????? /* 黑名单用户????? */
}MR_PassportErrorCode;
```

说明:

名称值说明 MR_PassportErrorCode_SUCCESS0 成功 MR_PassportErrorCode_UserCancel1 用户
取消 MR_PassportErrorCode_DIAL2 拨号错误 MR_PassportErrorCode_Network3 网络错误
MR_PassportErrorCode_InvalidUser4 无效用户 MR_PassportErrorCode_BlackListUser5
黑名单用户 ?

4.7???????????????? MR_GetPKList_CB

原型:

```
typedef void (*MR_GetPKList_CB) (MR_PKErrorCode errCode, MR_CharacterList*?lst);
```

说明:

mrc_netpk_getPKList 的回调函数。

4.8???????????????? MR_Netpk_upload_appdata_CB

原型:

```
typedef void (*MR_Netpk_upload_appdata_CB) (MR_PKErrorCode errCode, const char*  
respData, int32 len);
```

说明:

mrc_netpk_upload_appdata 的回调函数

4.9???????????????? MR_Netpk_commit_PKResult_CB

原型:

```
typedef void (*MR_Netpk_commit_PKResult_CB) (MR_PKErrorCode errCode, const char* respData, int32 len);
```

说明:

mrc_netpk_commit_pkresult 的回调函数

?

5. ?????????????????? 运行参数

联网计费插件资源的大小如下:

File	Name	Size (KB)	Origin	
inputbk.bmp	0.10.7	inputsele.bmp	0.10.7	
networkbk.bmp	0.10.2	networkslider.bmp	0.20.6	
buttoncom.bmp	0.40.7	buttonsele.bmp	0.30.7	
background.bmp	12.8	109.4	regbk.bmp	0.30.7
setcolumn.bmp	0.10.4	sex.bmp	2.47.0	
title.bmp	0.30.6	cancelconfirm.bmp	0.41.5	
passport.ext	18.3	28.9	netpk.ext	7.4
10.4	共计	43.2	162.5	?

这样 MRP 的包会多占用 43.2KB。

?

下面看一下插件运行时的内存峰值

因为使用了 dump 模式的主模块, 在实际测试中内存峰值并没有增加。

6. ?????????????????? 运行配置

6.1 ?????????????????? SDK 更新

将 联 网 PK 插 件 中 exttool 、 netpk 和 passport 放 到 C:\SKYMOBI\SDS4CPv1.0\compiler\plugins 目录下。

将 exttool 、 netpk 和 passport 目 录 中 的 *_win32.lib 放 到 C:\SKYMOBI\SDS4CPv1.0\simulator\lib 目录下。

?

6.2 ?????????????????? mpr 配置说明

为了简单起见, 第一期可以将所需要用到的插件 (PASSPORT 和 NETPK) 全部打入到游戏的 mpr 包。

?

此外还要注意, 由于 passport 插件是用 dump 模式加载的, 所以必须在打包 mpr 过程中, 必须注意使用 dump 版本的主模块。

?

6.3 ?????????????????? mpr 配置示例

应用 mpr 配置示例如下:

[information]

projectname=game.mpr

filename=game.mrp

appname=NETPK 测试

appid=335733

version=1010

visible=1

cpu=3

vendor=斯凯

description=斯凯应用插件示例--游戏 MRP

include= \${MRPBUILDER_HOME}\..\include, \${MRPBUILDER_HOME}\..\include\plugins

plugin=app

config=mtkt240, spt240

?

[config_mtk240]

plat=mtk

c_compiler=thumb

bmp_mode=normal

output= .\dist\netpk_tst_mtk.mrp

?

[config_spt240]

plat=spreadtrum

c_compiler=thumb

bmp_mode=spreadtrum

output= .\dist\netpk_tst_sp.mrp

?

[files_mtk240]

file01 = game_mtk.ext(name=game.ext)

file02 = \${MRPBUILDER_HOME}\plugins\mainext\mainext_mtk.d.mrp

file04 = \${MRPBUILDER_HOME}\plugins\verdload\verdload_mtk.mrp

file05 = \${MRPBUILDER_HOME}\plugins\passport\passport_mtk.mrp

file06 = \${MRPBUILDER_HOME}\plugins\netpk\netpk_mtk.mrp

?

?

[files_spt240]

file01 = game_sp.ext(name=game.ext)

file02 = \${MRPBUILDER_HOME}\plugins\mainext\mainext_sp.d.mrp

file04 = \${MRPBUILDER_HOME}\plugins\verdload\verdload_sp.mrp

file05 = \${MRPBUILDER_HOME}\plugins\passport\passport_sp.mrp

file06 = \${MRPBUILDER_HOME}\plugins\netpk\netpk_sp.mrp

?

?

[files_game_mtk]

file01 = dist\game_mtk.lib

file02 = \${MRPBUILDER_HOME}\plugins\verdload\verdload_mtk.lib

file03 = \${MRPBUILDER_HOME}\plugins\platreq\mr_platreq_mtk.lib

file05 = \${MRPBUILDER_HOME}\plugins\netpk\netpk_mtk.lib

file06 = \${MRPBUILDER_HOME}\plugins\passport\ passport_mtk.lib

?

[files_game_sp]

file01 = dist\game_spt.lib

file02 = \${MRPBUILDER_HOME}\plugins\verdload\verdload_spt.lib

file03 = \${MRPBUILDER_HOME}\plugins\platreq\mr_platreq_spt.lib

file05 = \${MRPBUILDER_HOME}\plugins\netpk\netpk_mtk.lib

file06 = \${MRPBUILDER_HOME}\plugins\passport\ passport_mtk.lib

?

[files]

6.4???????????? VC6 工程配置

相对于平常的功能来说，还应在原有基础上进行一些配置。具体如下：

选中 Project——>Setting，打开工程配置页面。

选中 C/C++选项页，在 Category: 中选中 Preprocessor，此时将出现如下界面，图 5.1。

图 5.1 Preprocessor 设置

?

在上图的 Additional include directories 栏中加入 mrc_netpk.h 以及 mrc_passport.h 所在目录。

在进行完上述配置后，选中 Link 选项页，在 Category: 中选中 Input。此时将显示如下界面，如图 5.2。

?

图 5.2 Input 设置

?

在 Object/library modules 中加入 passport_win32.lib、netpk_win32.lib、exttool_win32.lib、browser_win32.lib、platreq_win32.lib、netpay_win32.lib、verdload_win32.lib、mainext_win32.lib。在 Additional library path: 中加入 passport_win32.lib、netpk_win32.lib、exttool_win32.lib、browser_win32.lib、platreq_win32.lib、netpay_win32.lib、verdload_win32.lib、mainext_win32.lib 所在目录。

?

?

6.5???????????? VS2005 工程配置

1、?头文件路径添加

在 C:\SKYMOBI\SDS4CPv1.0\compiler\include\plugins 路径下添加 mrc_netpk.h 和 mrc_passport.h 头文件。

然后，在工程中按下图设置文件路径。

?

?

2、?lib 添加

在 C:\SKYMOBI\SDS4CPv1.0\simulator\lib 目录下添加下述 lib

passport_win32.lib、netpk_win32.lib、exttool_win32.lib、browser_win32.lib、platreq_win32.lib、netpay_win32.lib、verdload_win32.lib、mainext_win32.lib。

然后在工程中按下图设置 lib。

?

?

?

7.???????????? 注意事项

7.1???????????? 字库问题

因为之前有很多单机游戏中，为了解决 UI 排版的问题，使用了自带字库的方案，把游戏中所使用到的字符做成独立的 bitmap 文件和游戏打包在一起。??

?

由于在联网 PK 时，玩家的昵称等是不可预见的，所以采用自带字库的方式不能再解决问题。

？

解决方案有两种：

- 1) 游戏主体中还是使用自带字库，但是在某些场景下，使用手机自带的字库，例如在联网对战功能时使用手机自带字库。
- 2) 使用冒泡社区的小字体字库。缺点是对于没有安装过小字体的手机需要一次安装过程。需要下载 180K 的 mrc，外加安装过程。由于斯凯平台上网游的推广和普及，已经有一定比率（统计中，后续会更新数据）的终端已经安装了小字体。这个方案正在开发中，如果需要这个方案请联系我们，我们会加快开放速度。

7.2 模拟器问题

为了能正常的调试模拟器，需要注意以下几点

- 1、模拟器打开的 MRP 命名不宜过长，建议使用该游戏/应用的短名称；
- 2、模拟器打开的 MRP 目录不宜过深，建议直接 COPY 到 D 盘上再打开；
- 3、plugins 目录必须存在，和 MRP 文件在同一级目录下。

7.3 内存问题

在联网 PK 系列中下面的函数需要自己释放内存，所以必须要正确的释放要不然会造成内存泄露。

1、mrc_netpk_upload_appdata 函数的回调函数

```
/******  
/*mrc_netpk_upload_appdata 的回调原型
```

```
?errCode:?? 错误码????????????
```

```
?respData:?? 从服务端反馈的数据, 由应用负责释放, 目前为 utf16be 编码的文字??  
?len:????? respData 的长度???????????? */
```

```
/******
```

```
typedef void (*MR_Netpk_upload_appdata_CB) (MR_PKErrorCode errCode, const char*  
respData, int32 len);
```

其中的 respData 需要调用者使用 mrc_free 函数释放。

2、mrc_netpk_commit_pkresult 函数的回调函数

```
/******
```

```
/*mrc_netpk_commit_pkresult 的回调原型
```

```
?errCode:?? 错误码????????????
```

```
?respData:?? 从服务端反馈的数据, 由应用负责释放, 目前为 utf16be 编码的文字??
```

```
?len:????? respData 的长度????? */
```

```
/******
```

```
typedef void (*MR_Netpk_commit_PResult_CB) (MR_PKErrorCode errCode, const char*  
respData, int32 len);
```

其中的 respData 需要调用者使用 mrc_free 函数释放。

？

3、mrc_netpk_getPKList 函数的回调函数

```
/******
```

```
/* mrc_getPkList 的回调原型????????????????????
```

```
?errCode: 错误码????????????????????
```

```
?lst:??PK 角色列表, 仅当 errCode 为 MR_PKErrorCode_SUCCESS 时有
```

效.????????????????

???? ?lst 释放说明:

??????? ? lst 是一个链表结构, 最后一个节点指向 NULL。应用负责释放每个节点中 pCharacter 所指向的内存,

??????? ?以及 pNext 所指向的内存 (如果 pNext 不为 NULL)。特别注意, MR_Character 中 customData 指针所指向的

??????? ?内存也需由应用释放。

*/

/*****

typedef void (*MR_GetPKList_CB) (MR_PKErrorCode errCode, MR_CharacterList*?lst);

?

lst 释放代码如下:

???? MR_Character *pCharacter = NULL;

???? MR_CharacterList *pCharacterList = NULL;

???? MR_CharacterList *pCharacterListTmp = NULL;

???? pCharacterList = lst;

???? while (pCharacterList)

???? {

??????? pCharacter = pCharacterList->pCharacter;

?

??????? if (pCharacter->customData && pCharacter->dataLen > 0)

??????? {

??????????? mrc_free(pCharacter->customData);

??????? }

???????

??????? mrc_free(pCharacter);

??????? pCharacterListTmp = pCharacterList;

??????? pCharacterList = pCharacterList->pNext;

??????? mrc_free(pCharacterListTmp);